Final Report

By
Marie Ella Cambay
id: 40284457
Charles Dahan
id: 40263717
Ilyess Diyane
id: 40316188
Frederic Gagne
id: 40278058
Pablo Hernandez
id: 40246914
Quentin Malichecq-Laroche
id: 40285143

A Technical Report
submitted in partial fulfillment
of the requirements of SOEN 341
Concordia University April 2026

MealMajor Release Summary

THE COOL TEAM 😎

GitHub:

https://github.com/FredTheOkGuy/The_Cool_Team-SOEN341_Project_W26

1. Team members and contributions

A table tracking each member's story points, commits/PRs, and issues completed, used to verify individual accountability via GitHub stats.

Name & ID	GitHub ID	Number of story points completed	Number of commits	Number of issues completed
Marie Ella Cambay	marieellacambay	13	14	31
Charles Dahan	charlesdahan	18	19	20
Ilyess Diyane	Ilyessdyn	6	7	8
Frederic Gagne	FredTheOkGuy	27	58	41
Pablo Hernandez	PabloHatche	9	16	11
Quetin Malichecq-Laroche	wakefieldTemp	16	52	16
Total:		89 (shared)	166	127 (shared)

2. Project Summary

MealMajor is a meal planning web app designed specifically for students who struggle to stay on top of their nutrition. It allows you to create and manage your own recipe list, plan your meals across the week and track your calorie intake, all in one place. And when you're staring at a half-empty fridge with no idea what to cook, MealMajor's AI-powered recipe creator takes whatever ingredients you have on hand and generates a personalized recipe that respects your allergies and dietary preferences. It's the smart and simple way for students to plan their meals and to eat better without the stress.

3. Velocity and a list of user stories and non-story tasks for each iteration

A sprint-by-sprint breakdown of all user stories with their point values and completion status, linked to GitHub.

Total: 11 stories, 48 points over 11 weeks			
Iteration 1: 4 stories, 14 points over 2 weeks			
US #	US Name	Points	Status
1	When opening the website, the user is introduced to a login page.	2	Done
2	On the login page, the user must have the option to either login or create an account.	5	Done
3	The user's information is saved when registered.	3	Done
4	The user can manage his profile, with information like diet preferences and allergies.	4	Done
Iteration 2: 2 stories, 8 points over 3 weeks			
6	The user can access a page allowing to manage recipes (add, delete, modify, etc.).	5	Done
7	The user can look and filter through his recipes.	3	Done
Iteration 3: 3 stories, 19 points over 4 weeks			
5	When logged in, the user is introduced to the menu page with a user-friendly interface and the weekly meal schedule.	6	Done
8	The user can access a page where a calorie tracker advises him on his calorie consumption.	6	Done
9	The user can give all ingredients he has; the app will return him with suggestions of recipes that are possible with those ingredients	7	Done
Iteration 4: 2 stories, 5 points over 2 weeks			
10	The user will enjoy an easy to learn user interface with a fast response.	2	Done
11	The user can live comfortably knowing that his stored data is protected.	3	Done

4. Overall Architecture and Design

An MVC diagram illustrating how the Model, View, and Controller layers interact within the system, accompanied by a sequence diagram showing the flow of key operations between components.

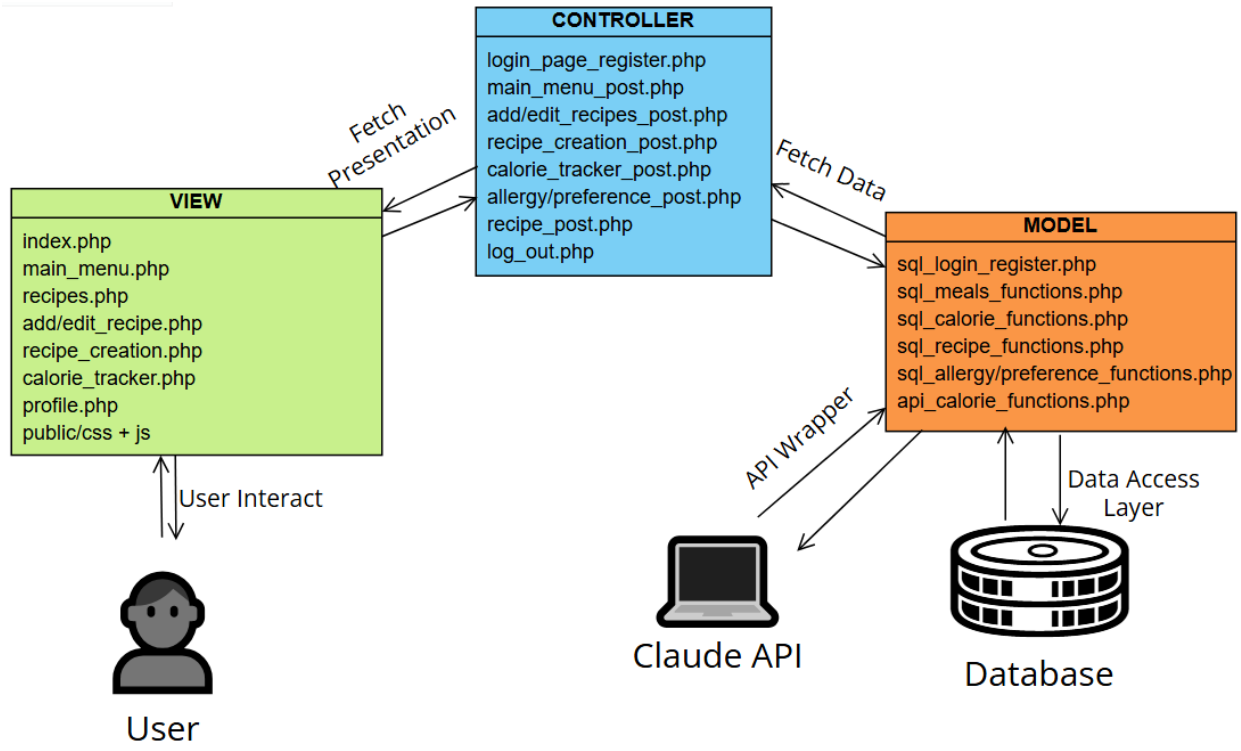

Figure 1: MCV Structure of MealMajor.

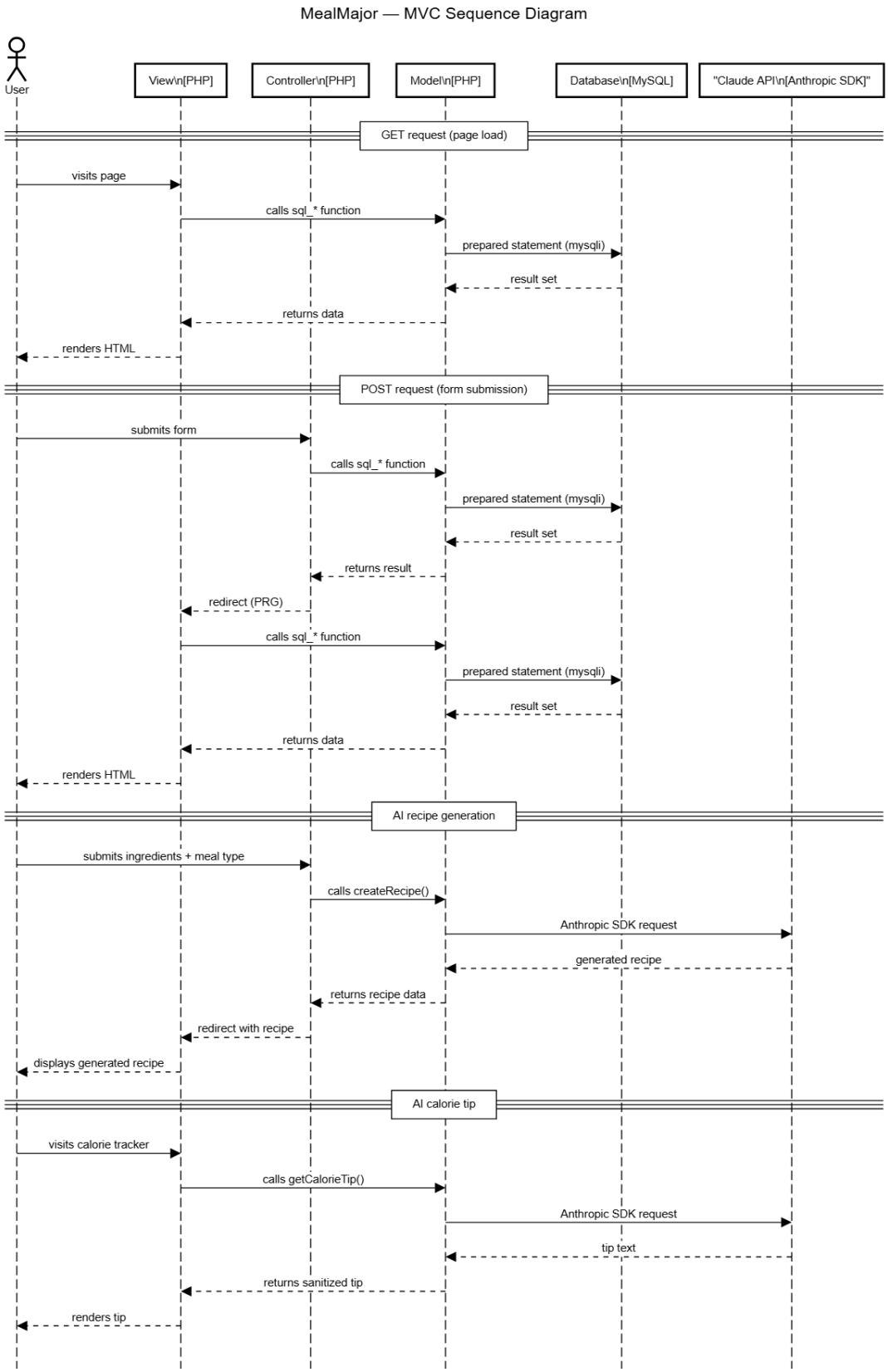

Figure 2: MealMajor's MVC Sequence Diagram.

5. Infrastructure

For each major tool or framework used, an explanation of why itw as chosen and what alternative were chosen.

1. **Paragonie | Anti-CSRF** https://github.com/paragonie/anti-csrf
   Why we need it: One of our user stories required a secure authentication system, which meant protecting our login and registration forms against Cross-Site Request Forgery (CSRF) attacks. The Paragonie Anti-CSRF library generates and validates unique tokens with minimal setup, integrating easily into our existing PHP session handling.
   Alternatives considered: We considered implementing CSRF tokens manually using PHP sessions, but rolling out our own security logic is more complicated and includes risks. Symfony's Security component also offers CSRF protection, but it would have introduced a far larger dependency than our project warranted.

2. **Anthropic | PHP SDK** https://platform.claude.com/docs/en/api/sdks/php
   Why we need it: Our AI recipe generator sends prompts to a language model and receives JSON responses based on the user's ingredients, allergies, dietary preferences, and meal type. Additionally, the calorie tracker offers encouraging messages and tips based on your calorie goal. The Anthropic PHP SDK handles authentication, request formatting, and response parsing, letting us focus on the prompt logic rather than raw HTTP calls.
   Alternatives considered: The most obvious alternative was the OpenAI SDK with a GPT model; however, we chose Anthropic primarily for ethical reasons. OpenAI has partnered with the Pentagon to have their AI used for mass surveillance and weapons systems, which conflicted with our values as a team. Given the choice between two technically comparable options, we felt it was more responsible to use a provider whose technology is not being applied in that context.

3. **PHPUnit** https://phpunit.de/index.html
   Why do we need it: Our project requirements include unit tests for all components, and PHPUnit is the standard testing framework for PHP. It allowed us to write tests for our controllers and models with minimal setup.
   Alternatives considered: We considered Pest, a more modern PHP testing framework built on top of PHPUnit with a cleaner syntax. However, PHPUnit has significantly more documentation and community support, making it easier to work with, and it is the framework most used in PHP development.

4. **MySQL** https://www.mysql.com/
   Why we need it: MySQL is used to store and manage all data in our application, including user accounts, recipe information, and calories. It provides a reliable relational database

structure that allows us to define clear relationships between data entities and perform efficient queries in PHP.

Alternatives considered: We considered MongoDB, a NoSQL option that provides more flexibility with unstructured data, and PostgreSQL, which offers more advanced features. We chose MySQL because of its simplicity, large amount of documentation, and ease of integration with our backend.

5. **GitHub** https://github.com/

   Why we need it: GitHub is used as our version control and collaboration platform. It allows all team members to work on the codebase at the same time without conflict. It also hosts our project issues, pull requests, sprint tracking, and CI, making it central to our agile workflow.

   Alternatives considered: We considered SVN, a centralized version control system that tracks changes to files over time. However, unlike GitHub's distributed model, SVN requires a central server for most operations, making collaboration less flexible. We chose GitHub as required by the course, and its distributed nature, larger community support, and built-in issue tracking made it the best option for our team.

6. Name Convention

A list of the coding style and naming standards the team followed, with what they were used for, examples, and with reference links.

Naming Convention	Used For	Examples
snake_case	Website File Names Variable Names	recipe_creation.php $meal_result
camelCase	Test File Names Function Names	createRecipeTest.php addRecipe()
kebab-case	HTML Class Names	form-box
SCREAMING_SNAKE_CASE	API Variable Names File Path Variable Names	$ANTHROPIC_API_KEY BASE_URL

7. Code

The five most critical files in the codebase, with links and a one-line description of each file's purpose.

File Path	Purpose
Index.php	Entry point of the application; renders the login and registration forms, handles CSRF token injection, and displays session-based error and confirmation messages.
Sql_recipe_functions.php	Model layer for all recipe-related database operations, including adding, editing, deleting, fetching, and displaying recipes, as well as the AI-powered recipe generator using the Anthropic Claude API.
Login_page_config.php	Global configuration file that dynamically defines BASE_URL and establishes the MySQL database connection.
Main_menu.php	Main dashboard view that displays the user's weekly meal schedule with the ability to add and remove meals per day, and provides navigation to recipes, the calorie tracker, the recipe creator and your profile.
Login_page_register.php	Controller that handles login and registration form submissions, including CSRF validation, password hashing and verification, and redirecting with error or confirmation messages.

8. Testing and Continuous Integration

The top unit and acceptance tests, an overview of the CI-pipeline, the static analysis tool used, and a bug-fix maintenance report in the Appendix.

The login and registration features do not have automated unit tests, as they are heavily dependent on database connections and session handling, making them difficult to test in isolation. Instead, their quality was verified through manual acceptance testing, where testers confirmed that valid credentials grant access. Invalid credentials are rejected and duplicate registrations are prevented.

Unit Test	
File Path	**Testing Objective**
CreateRecipeTest.php	Tests that the AI recipe generator correctly decodes ingredients, calls the API with the right arguments, and saves the result with normalized values.
DeleteRecipeTest.php	Tests that deleting a recipe prepare the right SQL, bind the correct parameters, and don't fire at all when the delete flag is missing.
EditRecipeTest.php	Tests that the edit recipe redirects when no recipe ID is given, pre-populates the form correctly, and saves proper values.
SearchRecipeTest.php	Tests that the recipe list query is built correctly based on whatever search terms, filters, and sort options the user provides.
WeeklyPlanTest.php	Tests that the meal schedule correctly adds meals, builds the weekly grid, loads the recipe dropdown, and blocks duplicate entries.

Acceptance Test	
File Path	**Tested User Story**
AT1	US.02 On the login page, the user has to have the option to either login or create an account.
AT2	US.04 The user is able to manage his profile, with information like diet preferences, and allergies.
AT3	US.05 The user can access a page allowing to manage recipes (add, delete, modify, etc.).
AT4	US.06 The user can look and filter through his recipes.
AT5	US.09 The user can give all ingredients he has; the app will return him with suggestions of recipes that are possible with those ingredients.

Continuous Integration Pipeline

Link to the continuous integration environment

For the Continuous Integration Pipeline, we use PHPUnit as the unit testing framework, seeing as it is the default choice for our use case due to reliability and ease of use. The pipeline itself runs on the latest version of Ubuntu, which at the time of writing this report is 24.04, and runs a Docker container with a local MySQL database to assist with the tests. The reason that we use a local database for unit testing is that it was found to be more convenient to extract our functions' results from a database rather than from function return statements. The tradeoff for this decision is a slower execution of the CI due to the containers and MySQL's startup times. The CI's features include project wide syntax analysis and dependency installation, as well as running the unit tests found here.

Static Analysis

For static analysis, the team chose SonarQube, which natively supports PHP, the main language used in the bast majority of the source code. It was integrated directly into the GitHub Actions CI pipeline, via a dedicated sonarcloud.yml workflow. This means that analysis runs automatically on every push and pull request without any manual steps. SonarQube is preferred over PHP-specific tools like PHPStan or Psalm, because it combines static analysis, security vulnerability detection, and code quality metrics, all in one tool. Additionally, the results are easily accessible through the SonarQube dashboard. The static analysis tool identified many bugs, some that were addressed and resolved. The database password was hardcoded in the configuration file and was moved to a .env file to prevent credentials from being exposed in the repository. Multiple if, while and foreach statements were missing curly brackets, which were added to prevent unexpected behavior. Additionally, a repeated hardcoded path /src/views/recipes.php that appeared three times in the same file was refactored into a single reusable variable.

As shown in Figure 3 in the Appendix, the dashboard before the fixes displayed a Security rating of E with 1 open issue and 99 Maintainability issues. After applying the fixes, Figure 4 in the Appendix shows the Security rating improved to A with 0 open issues, and Maintainability was reduced to 82.

Appendix

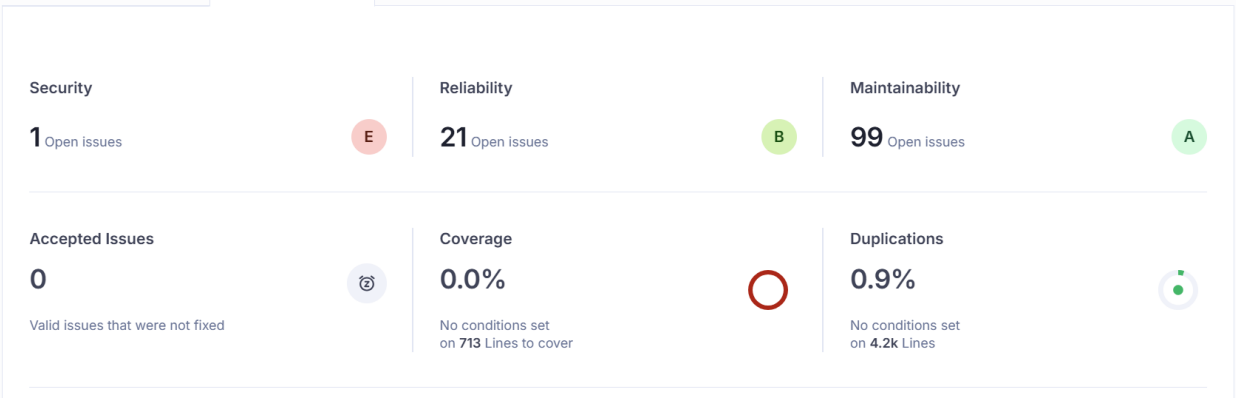

Figure 3: SonarQube Dashboard Before some Fixes.

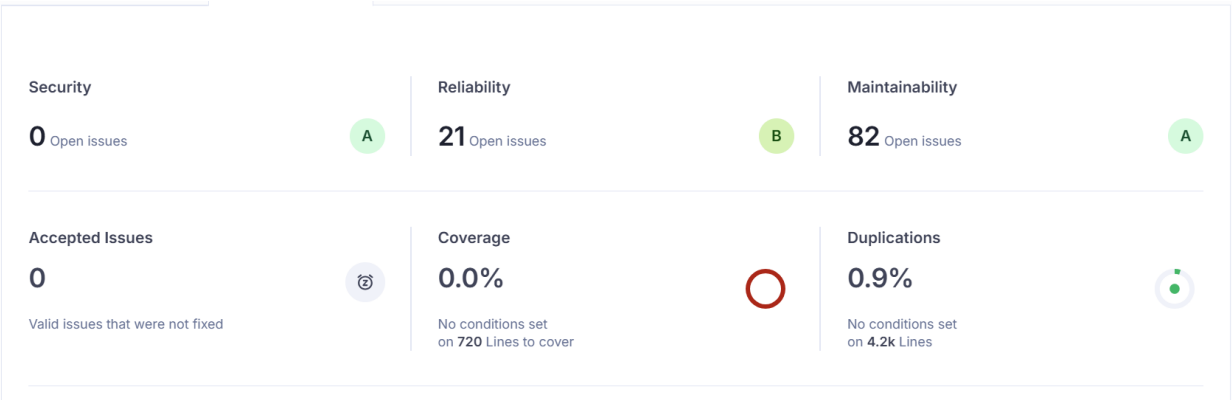

Figure 4: SonarQube Dashboard After the Fixes.